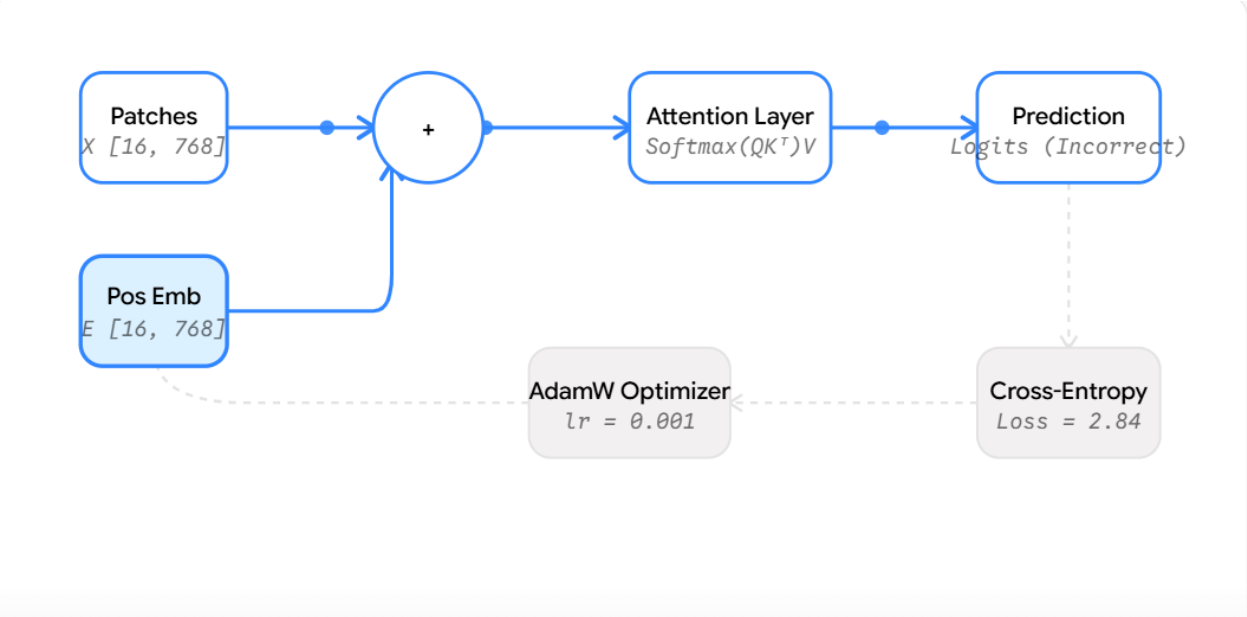
Here is the exact lifecycle of a learnable positional embedding:

1. **Initialize with random numbers.** You create a table of vectors. If your image splits into 196 patches plus one classification token, you create 197 vectors. You fill these vectors with random Gaussian noise. At step zero, the model has no concept of space.
2. **Add them to your patches.** You add Positional Vector 1 to Patch Embedding 1. You add Positional Vector 2 to Patch Embedding 2. You feed this combined sequence into the Transformer.
3. **Calculate the error.** The model makes a prediction. It might misidentify a picture of a car. You calculate the loss to measure how wrong the prediction is.
4. **Update the positions via backpropagation.** You calculate gradients for every parameter in the model. This includes your positional embeddings. You update these position vectors slightly to minimize the error.
5. **Repeat millions of times.** The model processes millions of images. It continuously tweaks the positional vectors. It discovers that certain patches always appear next to each other. For example, it learns that a wheel patch usually sits below a window patch.

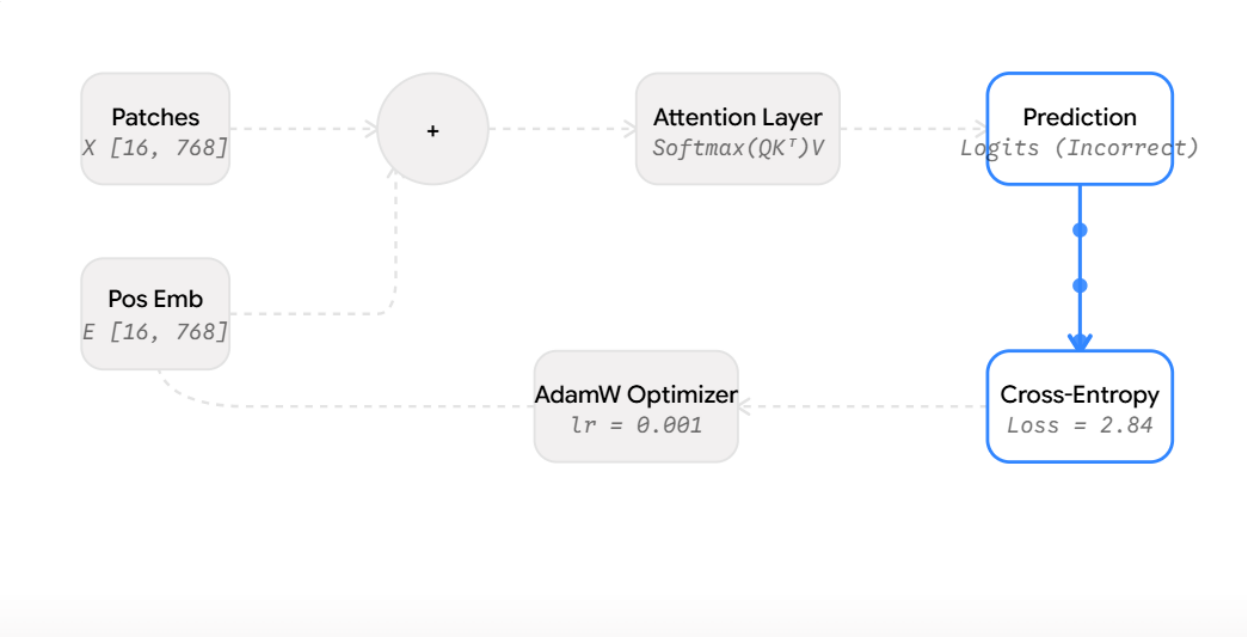
1. Forward Pass (Data Flow)
$E_{pos}[0]$:[+0.42, -0.18, +0.85]
Visual patches are combined with **Positional Embeddings**. The composite sequence flows through Self-Attention to generate class logits.



2. Loss Calculation

$E_{pos}[0]$:[+0.42, -0.18, +0.85]

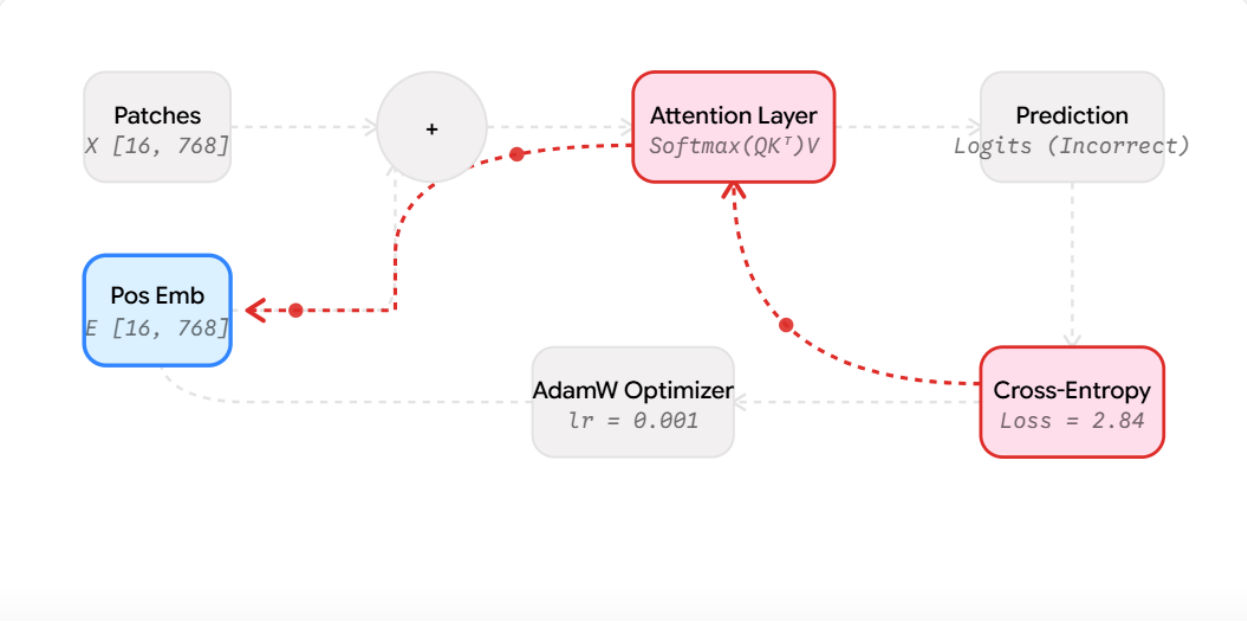
Prediction is evaluated against ground-truth label using Cross-Entropy Loss. A high loss of **2.84** reflects incorrect spatial patch layout.



3. Backward Pass (Gradient Flow)

$E_{pos}[0]$:[+0.42, -0.18, +0.85]

Error gradient $\partial L/\partial E_{pos}$ flows backward through Attention Layer down to Positional Embeddings. Red dashed arrows depict gradient flow.



4. Optimizer Parameter Update

$E_{pos}[0]$:[+0.38, -0.12, +0.79] Loss:0.41 (Updated)

AdamW applies parameter step: $\mathbf{E_{new} = E_{old} - lr \cdot \hat{m}/(\sqrt{\hat{v}} + \varepsilon)}$. Positional embedding values are updated to reduce loss.

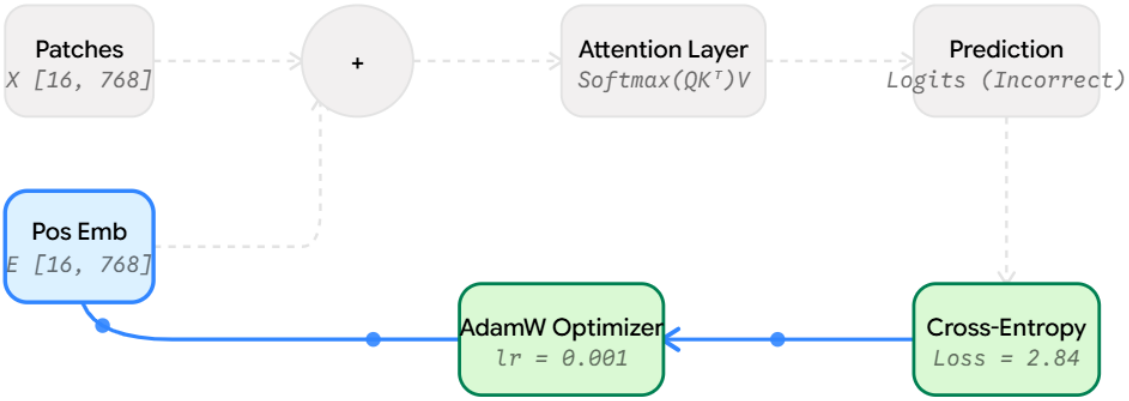
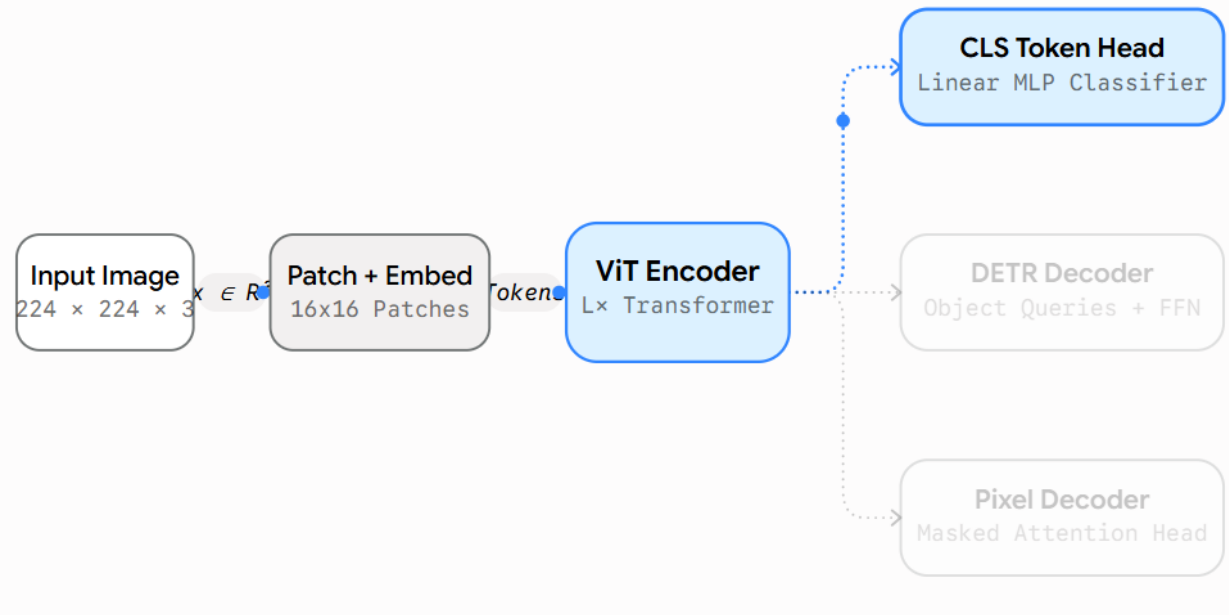


Image Classification (CLS Token)

`Output: [B, 1000]`

A prepended `[CLS]` token aggregates global context across all patch tokens via Self-Attention layers. A lightweight linear MLP converts the final `[CLS]` state directly into target class probabilities.

`Image [224, 224, 3]` → `Patches [197, 768]` → `CLS Token [768]` → `Logits [1000]`

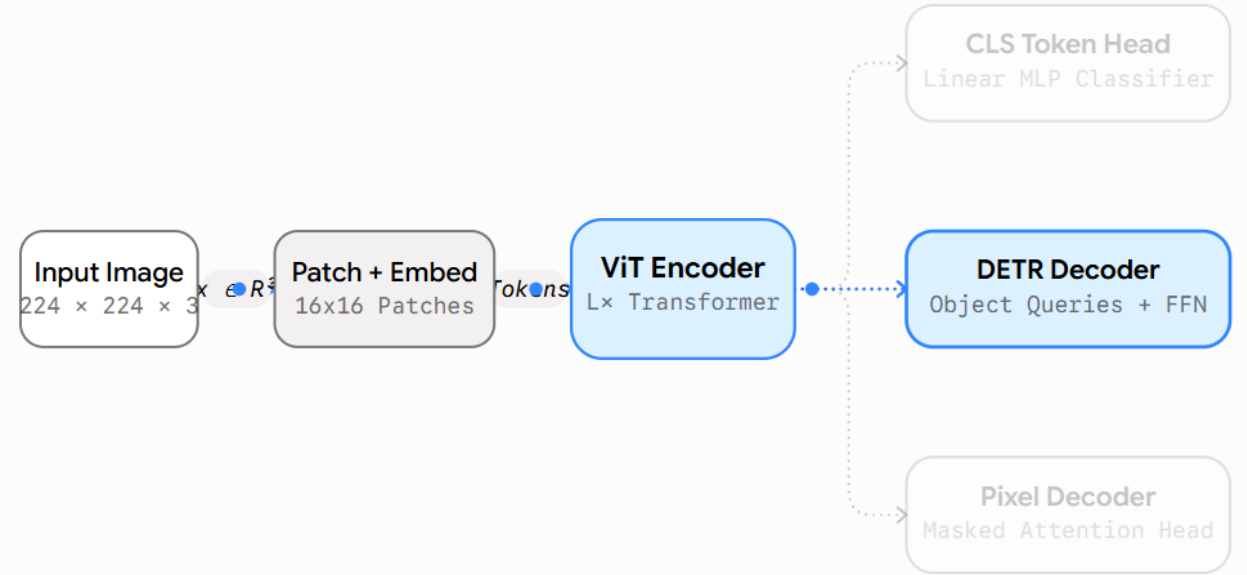


Object Detection (DETR Architecture 'DEtection TRansformer')

`Output: [B, N, 6]`

ViT feature maps feed into a Transformer Decoder with learned `Object Queries`. Cross-attention layers query spatial feature tokens, yielding bounding box coordinates `(x, y, w, h)` and class labels per query slot.

`Image [224, 224, 3]` → `Feature Map [196, 768]` → `Decoder Queries [100, 256]` → `Boxes + Classes [N, 6]`

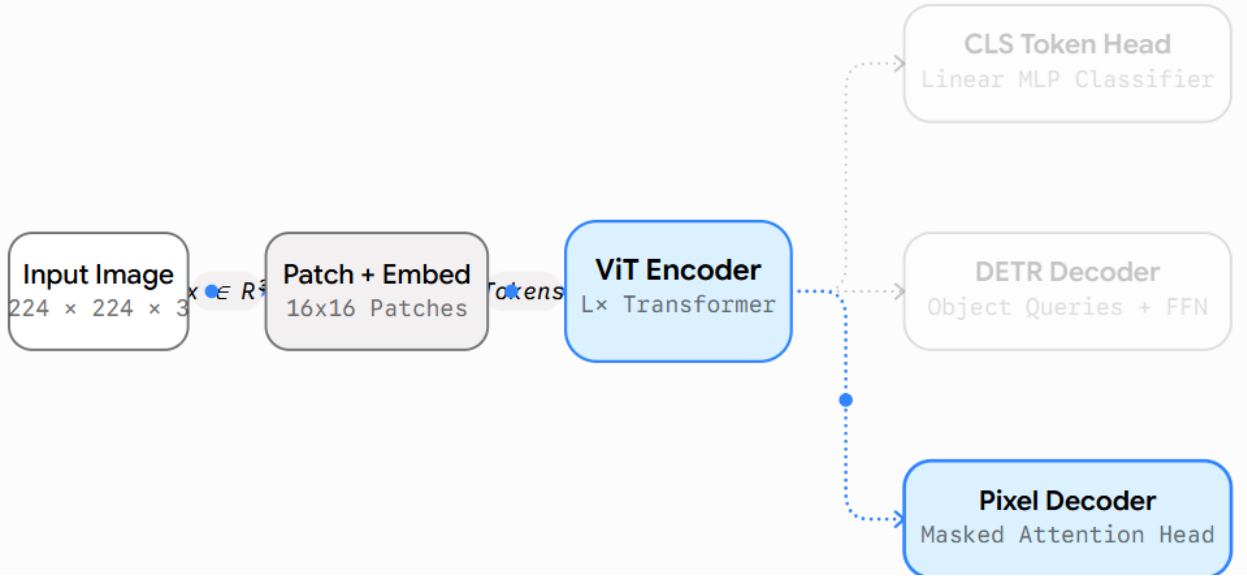


Semantic / Panoptic Segmentation (Mask2Former)

`Output: [B, N, H, W]`

Multi-scale features from ViT are decoded by a Pixel Decoder. Masked Attention restricts query attention to predicted mask regions, generating high-resolution binary pixel masks paired with class predictions.

`Image [224, 224, 3]` → `Multi-scale Feats` → `Pixel Decoder [H/4, W/4]` → `Mask Embeddings [N, H, W]`



CNN & Vision Transformer Hybrid Architectures

